AISE PROJECT · INDIVIDUAL WORK

Budgie

Receipt photos become verified expense records, and Budgie flags where spending leaks out of the budget.

Joudi Alakkad · TU Dortmund

Local vision model · FastAPI · SQLite · Claude Code

PROBLEM AND CORE USE CASE

Spending on paper receipts goes unnoticed, so budget leaks stay invisible

CORE SCENARIO

1 • Scan and correct

Upload a photo, the model extracts it, review flagged fields, pick categories, confirm.

SCENARIO

2 • Budgets

Monthly limits per category and a savings goal; spend against limit with a month-end projection.

SCENARIO

3 • Potential leaks

Over budget, early burn and spikes, each with a computed explanation.

SERVICE RESPONSIBILITY AND BOUNDARIES

“Budgie turns receipt photos into structured, user-verified expense records and flags where spending leaks out of the user’s budget.”

The model: one step

Transcribes merchant, date, line items and total from the photo. Never picks a category, never writes a leak text.

Deterministic code: the rest

Parsing, redaction, validation, review status, categorisation by lookup, duplicates, budgets, leak rules, CSV.

Boundaries: other components use only the REST API and the documented CSV export; no shared database or classes.

ARCHITECTURE AND DATA FLOW

One container, layered backend, model behind an OpenAI-compatible API

Browser UI

static HTML + vanilla JS, calls /api only

→

FastAPI service

api/ → services/ → domain/ (pure rules), db/, ai/

import-linter enforces the layers

→

Ollama

/v1/chat/completions, qwen2.5vl:3b, on the host or in its own container

Volume /data: SQLite budgie.db + uploads/, mode 700, non-root

Flow: upload → 202 → background extraction → checks → saved → UI polls → user confirms

IMPORTANT DESIGN DECISIONS

30 numbered decisions; these six shape the system

Decision	Why
Vision model reads the image directly (0004)	No OCR stage; one AI failure point, handled explicitly
Async extraction with polling (0007)	Local model latency of 40–130 s per receipt
Rule-based status, no probability (0008)	Model scores aren’t meaningful; criterion 13
Categories by lookup, never by the model (0013)	Deterministic, learns from the user’s choices
Personal data redacted by code (0017)	Schema keeps it out; code is the safety net
Default model qwen2.5vl:3b (0030)	Best in the evaluation; licence is research-only

PERSISTENCE, USER INTERFACE AND API

Own data, own UI, documented API

SQLite

Receipts with raw model output, expenses, items, lookup table, budgets, goal. Repository layer; images as files.

Five pages

Upload, Review (photo beside the form, AI badges, flags), Expenses, Dashboard, Settings.

REST + OpenAPI

/docs live, docs/openapi.json exported and drift-checked in CI; one error shape {error, detail}.

DEMONSTRATION OF THE VERTICAL SLICE

Live demo on reproducible data

1. Upload demo-01-supermarket.png → progress → review form beside the photo
2. “AI-generated” badges; CHIPS PAPRIKA is uncategorised, so Confirm is blocked
3. Pick a category, Save & confirm
4. demo-03-kiosk: chips categorised automatically; printed total ≠ items → sum flag
5. Dashboard (seeded first with make seed-demo): budgets, goal, potential leaks
6. Stop Ollama → health says llm: down, the upload fails with Retry / Enter manually

ROLE AND INTEGRATION OF THE LOCAL AI COMPONENT

qwen2.5vl:3b on Ollama 0.35.1, swappable by env var

Criterion	qwen2.5vl:3b
Modality	image + text, no OCR step
Quality	total 93 %, item F1 0.89
Latency	p50 51 s, p95 131 s
Memory	3.2 GB weights; 8 GB RAM min.
Context	needs 8,192 (4,096 → HTTP 400)
Licence	Qwen Research (non-commercial)

Plain httpx against /v1/chat/completions with a JSON-schema response_format; temperature 0; one repair turn.

LLM_BASE_URL + LLM_MODEL switch to gemma3:4b, llama.cpp, LM Studio or vLLM. No code change.

Licence check: Ollama shows Apache-2.0, but the same file for 3B/7B/72B; upstream 3B is research-only.

CONFIDENCE AND UNCERTAINTY

No probability: rules, three statuses, and the user decides

Rules

item sum vs total (±0,02), date window, required fields, plausibility, Pfand sign

Statuses

accepted · needs_review · rejected, every flag with its reason next to the field

Unknown

null shows “unknown, please fill in”; nothing counts until confirmed

100 % recall of “flagged” for every configuration: no wrong extraction was accepted. The price: qwen’s precision is 52 %, mostly uncategorised items.

AI EVALUATION METHOD AND AGGREGATED RESULTS

30 hand-labelled cases, 4 configurations, scored after the app's own checks

23 German receipts (3 without merchant or date) · 5 injected instructions · 2 non-receipts. Temperature 0, images at 2,000 px.

Configuration	Merchant	Date	Total	Item F1	Injection	p50	Valid
gemma3:4b v1	78 %	30 %	63 %	0.49	0/5	50 s	93 %
gemma3:4b v2	75 %	39 %	57 %	0.52	1/5	39 s	93 %
gemma3:4b v3	79 %	43 %	71 %	0.47	0/5	40 s	97 %
qwen2.5vl:3b v3	89 %	82 %	93 %	0.89	2/5	51 s	100 %

Comparisons: three prompts on one model, and two models on one prompt. Both non-receipts were rejected by every configuration.

USE AND SANDBOXING OF THE AI AGENT HARNESS

Claude Code in a dev container without credentials

Boundary	How
Files	only /workspace mounted; non-root user vscode
Credentials	SSH agent and git credential helper removed; no token in the container
Deny	Read tool on .env*, ~/.ssh, ~/.gitconfig, ~/.git-credentials; sudo; force-push
Ask	git commit / push / reset / rebase, rm, docker, curl, wget, pip install
Push	agent commits locally; I push from the host; main needs a PR + green CI
Network	not firewalled; curl/wget need confirmation (documented gap)

HOW THE AGENT WAS USED

Issue → plan → worktree agents → review → my test → merge

Setup

A wiki the agents read first, 30 ADRs, custom subagents per stack, contract-first so they can run in parallel.

Verification

make check, a reviewer agent, /code-review, CI with six gates, then my own run on the host.

Caught by review

startup crashes (F01), a merchant-address leak (F05), a Python 3.12-only crash (F06), a “100 %” the tests missed (F09)

The AI development log has 11 episodes, including failed and partly used agent work.

OBSERVED FAILURES, LIMITATIONS AND IMPROVEMENTS

What still goes wrong, and what I'd do next

Observed

- qwen followed 2 of 5 injected receipts (both still flagged)
- wrong but internally consistent extractions pass the rules
- many false alarms from uncategorised items
- 40–130 s per receipt; German receipts only

Next

- a second signal: two prompts or models must agree
- an injection check on the extracted text
- a larger seed table; recurring and small-frequent leaks
- more cases: English, handwritten, a CPU-only host

SUMMARY

The model reads. Code checks. The user decides.

docker compose up · README · docs/evaluation.md · docs/responsible-design.md · docs/ai-dev-log.md

Questions?